

AVL Ağaçları: Motivasyon

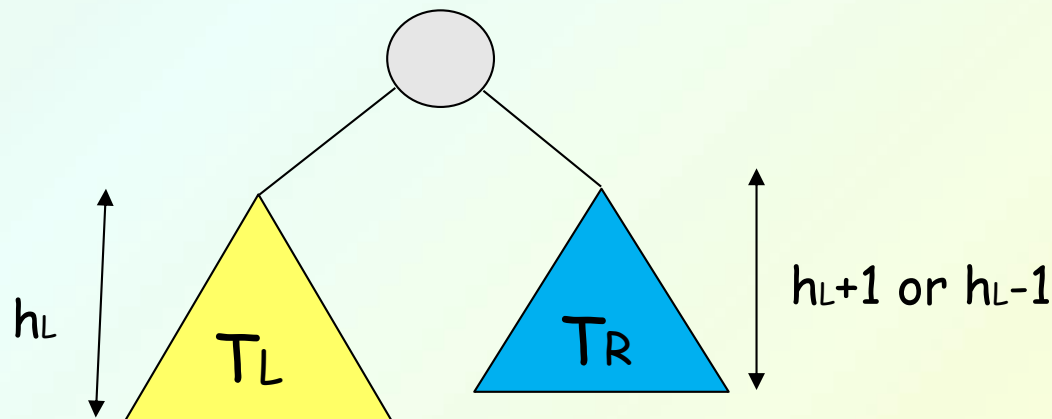
- İkili arama ağaçları üzerindeki tartışmalarımızı hatırlarsak
 - Ağacın yüksekliği ekleme sırasına bağlıdır.
 - Ö.g., 1, 2, 3, 4, 5, 6, 7 sayılarını boş bir ağaca eklersek
 - Problem: Dengenin bozulması - Ağaç bağlantılı liste şeklinde bozulur.
 - Tüm operasyonların karmaşıklığı $O(h)$ olduğunda, $\log N \leq h \leq N-1$, İkili arama ağacı operasyonlarının en kötü çalışma zamanı $O(N)$ olur.
- Soru: Ekleme sırasına bakmadan BST nin yüksekliği $\log(n)$ olabilir mi? Bir başka deyişle, BST'yi dengeli bir şekilde tutabilir miyiz.

Yükseklik Dengeli Ağaçlar

- BST operasyonlarını daha kısa sürede gerçekleştirmek için pek çok BST dengeleme algoritması vardır.
 - Adelson-Velskii ve Landis (AVL) ağaçları (1962)
 - Splay ağaçları (1978)
 - B-ağacı ve diğer çok yönlü arama ağaçları (1972)
 - Red-Black ağaçları (1972)
 - Ayrıca Simetrik İkili B-Ağaçları(Symmetric Binary B-Trees) şeklinde de bilinir

AVL Ağaçları: Tanım

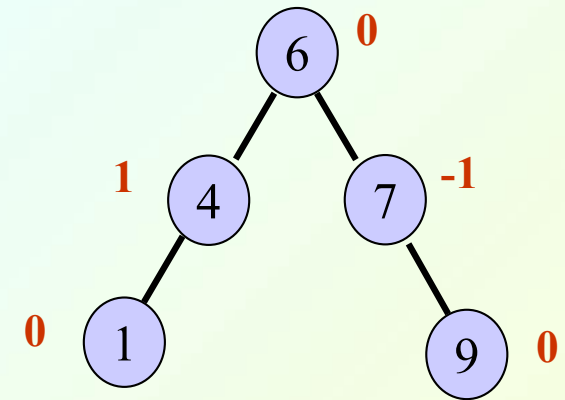
1. Tüm boş ağaç AVL ağacıdır.
2. Eğer T boş olmayan T_L ve T_R şeklinde sol ve sağ alt ağaçları olan ikili arama ağacı ise, T ancak ve ancak aşağıdaki şartları sağlarsa AVL ağacı şeklinde isimlendirilir.
 1. T_L ve T_R AVL ağaçları ise
 2. h_L ve h_R T_L ve T_R nin yükseklikleri olmak üzere $|h_L - h_R| \leq 1$ olmak zorundadır.



AVL Ağaçları

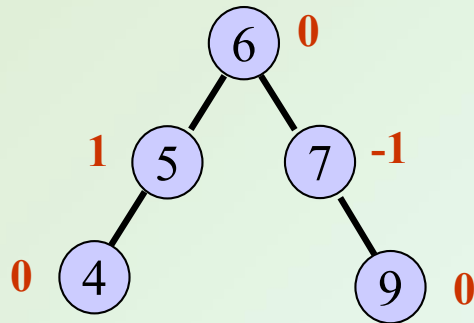
- AVL ağaçları dengeli ikili arama ağaçlarıdır.
- Bir düğümdeki denge faktörü = $\text{yükseklik}(\text{sol altağaç}) - \text{yükseklik}(\text{sağ altağaç})$
- AVL ağaçlarında balans faktörü sadece -1, 0, 1 olabilir.
- Her bir düğümün sol ve sağ alt ağaçlarının yükseklikleri arasındaki fark en fazla 1 olabilir.

AVL Ağacı

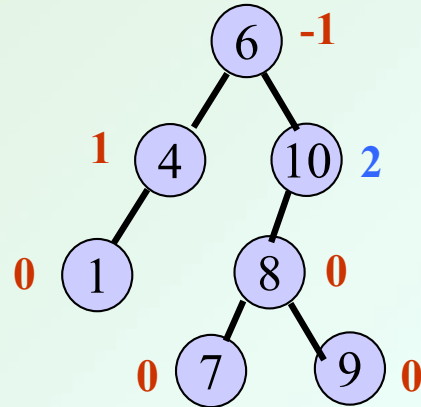


Kırmızı numaralar balans faktörüdür.

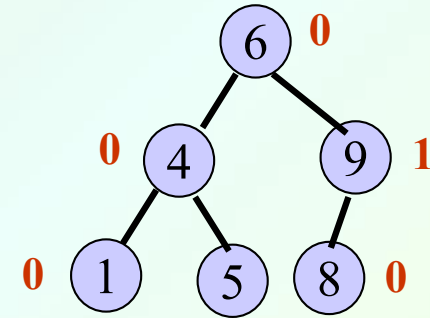
AVL Ağaçları: Örnekler



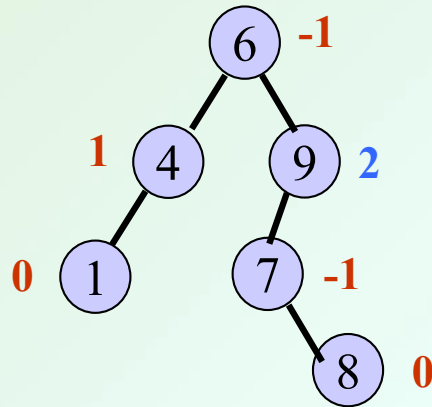
AVL Ağacı



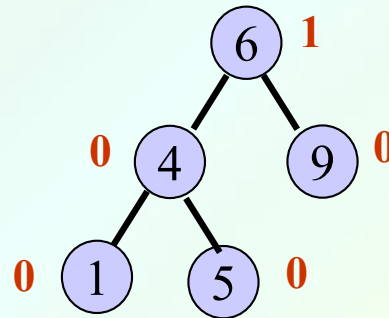
AVL Ağacı Değildir



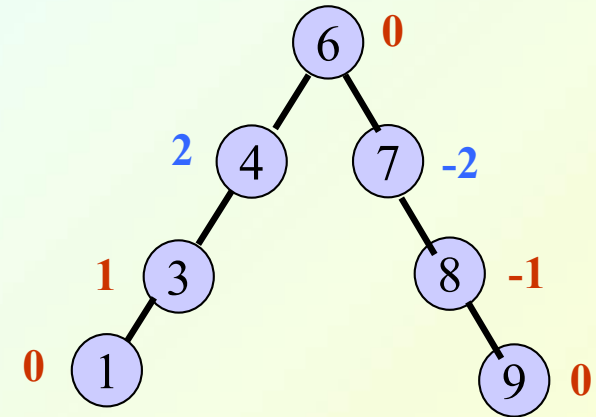
AVL Ağacı



AVL Ağacı Değildir



AVL Ağacı

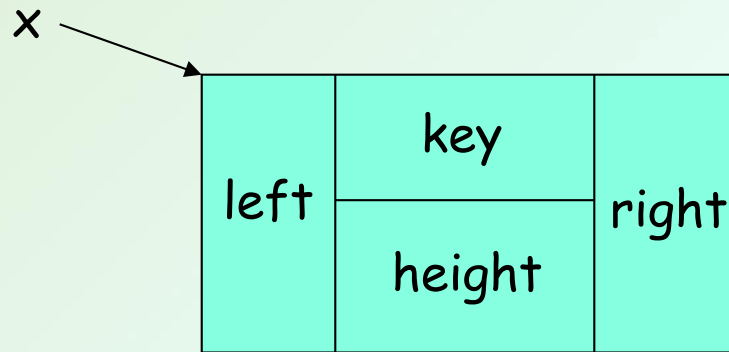


AVL Ağacı Değildir

Kırmızı numaralar balans faktörüdür.

AVL Ağacı: Gerçekleştirim

- AVL ağacının gerçekleştirimi için her x düğümünün yüksekliği kaydedilir.



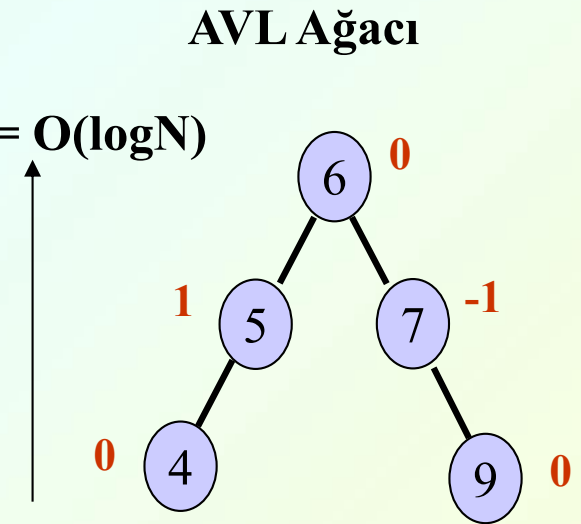
```
class AVLDugumu{  
    int deger;  
    int yukseklik;  
    AVLTdugumu sol;  
    AVLDugumu sag;  
}
```

- x 'in balans faktörü = x 'in sol alt ağacının yüksekliği - x 'in sağ alt ağacının yüksekliği
- AVL ağaçlarında, "bf" sadece $\{-1, 0, 1\}$ değerlerini alabilir.

AVL Ağaçları

- N düğümlü bir AVL ağacının yüksekliği daima $O(\log N)$ dir.
- Peki nasıl?

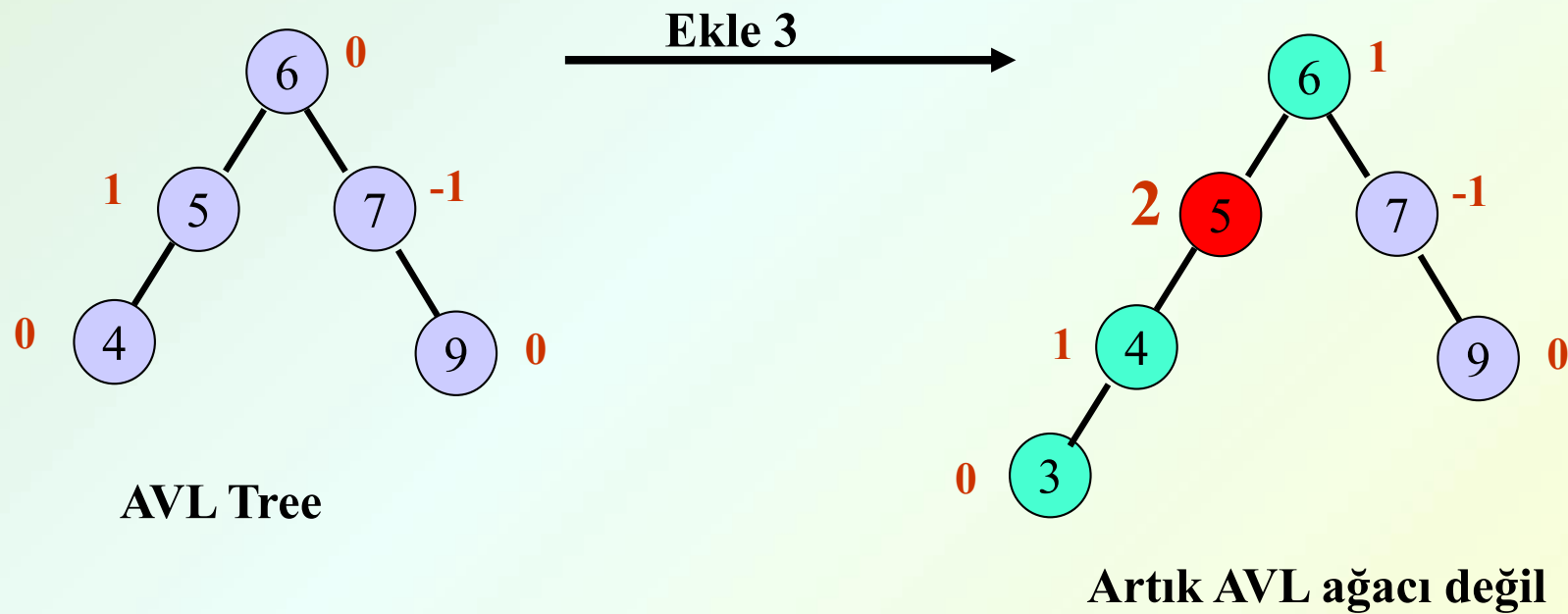
Yükseklik = $O(\log N)$



Kırmızı numaralar balans faktörüdür.

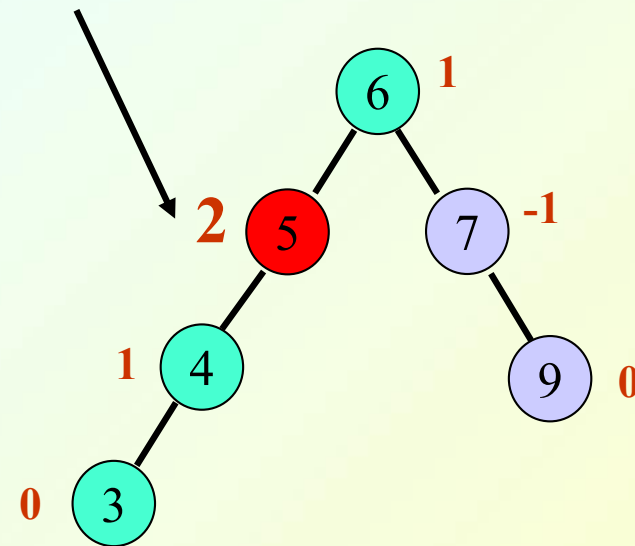
AVL Ağaçları: Güzel ve Çirkin

- Güzel:
 - Arama süresi $O(h) = O(\log N)$
- Çirkin
 - Ekleme ve silme işlemleri ağacın dengesiz olmasına neden olabilir.

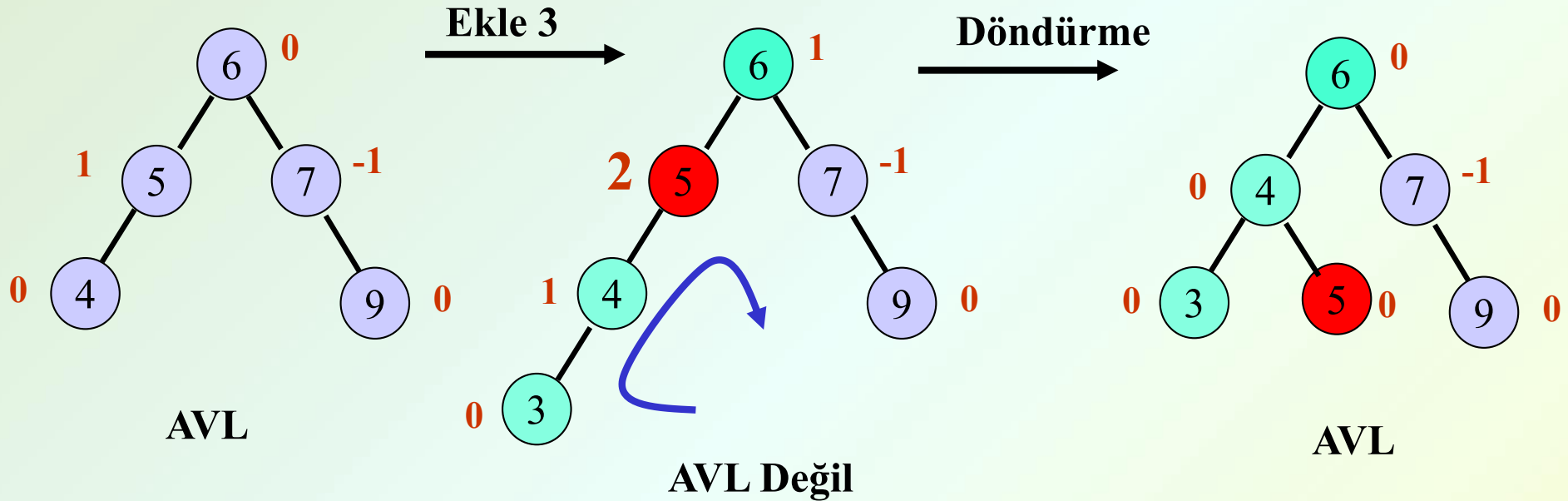


AVL Ağacında Dengenin Sağlanması

- **Problem:** Ekleme işlemi bazı durumlarda **ekleme noktasına göre kök olan bölgelerde** balans faktörün 2 veya -2 olmasına neden olabilir.
- **Fikir:** Yeni düğümü ekledikten sonra
 1. Balans faktörü düzelterek köke doğru çık.
 2. Eğer düğümün balans faktörü 2 veya -2 ise ağaç bu düğüm üzerinde döndürülerek düzeltilir.

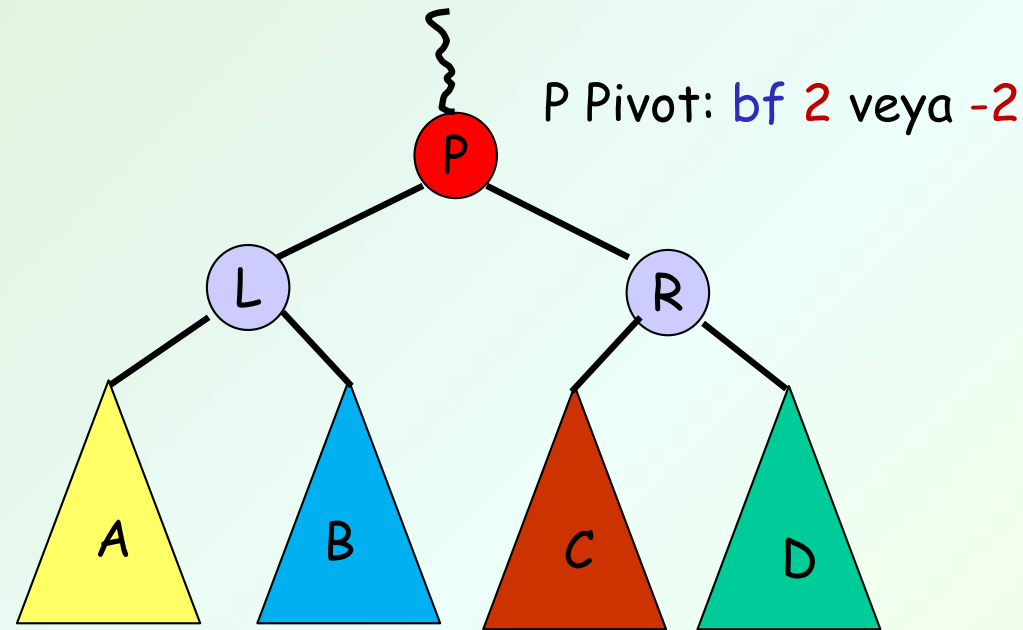


Denge Sağlama: Örnek



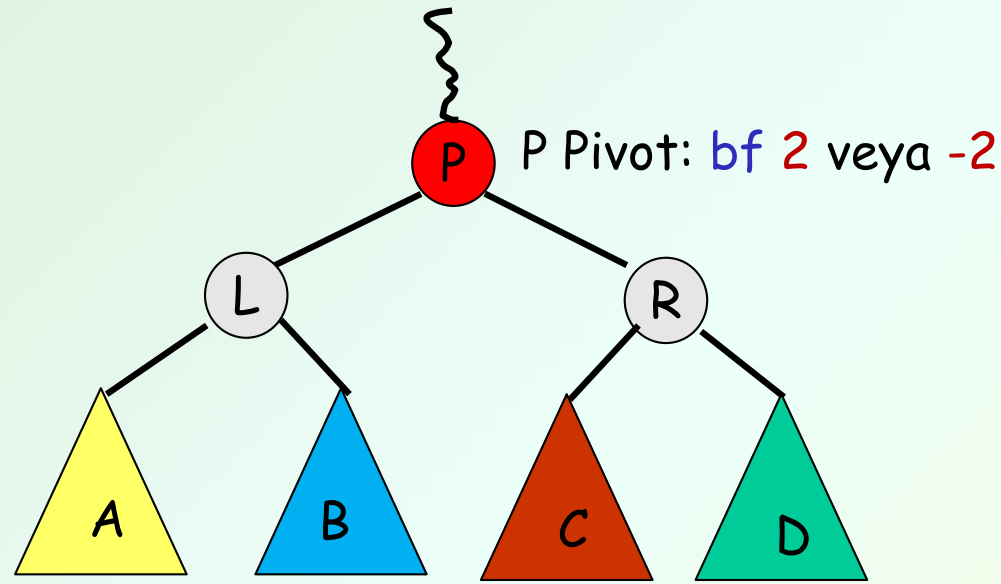
- Yeni düğümü ekledikten sonra
 1. Balans faktörü düzelterek köke doğru çık.
 2. Eğer düğümün balans faktörü 2 veya -2 ise ağaç bu düğüm üzerinde döndürülerek düzeltilir.

AVL Ağacı - Ekleme (1)



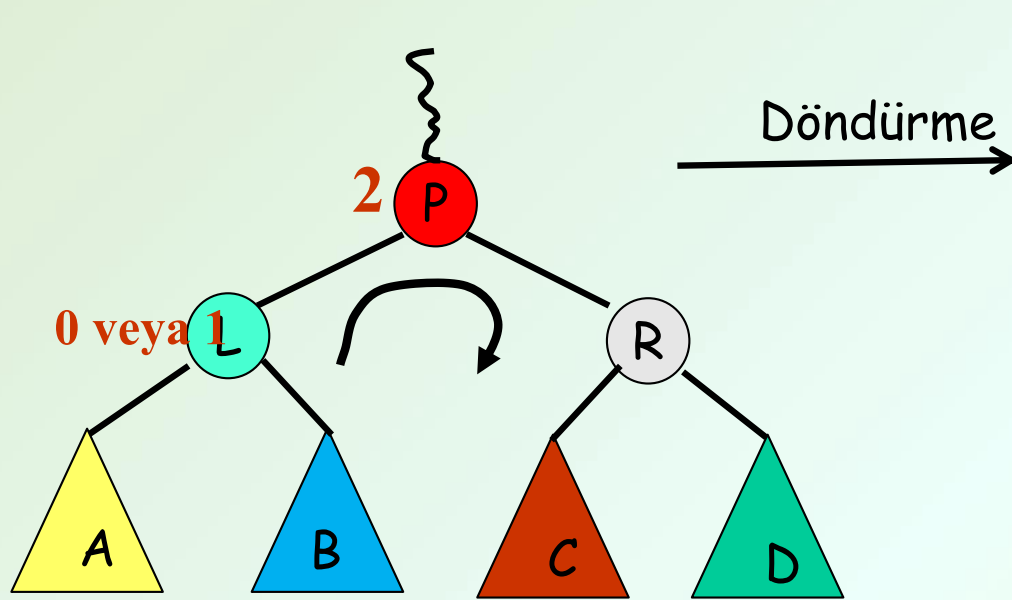
- **P** düğümünün dengeyi bozan düğüm olduğu düşünülürse.
 - **P pivot** düğüm şeklinde isimlendirilir.
 - Eklemeden sonra köke doğru çıkarken bf'nin 2 veya -2 olduğu ilk düğümdür.

AVL Ağacı - Ekleme (2)

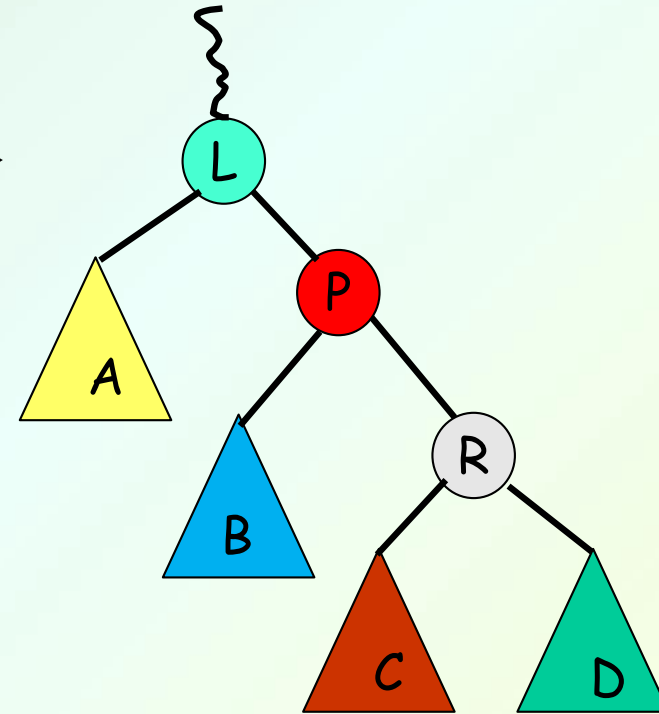


- 4 farklı durum vardır:
 - **Dış Durum** (tek döndürme gerektiren) :
 1. P'nin sol alt ağacının soluna eklendiğinde (LL Dengesizliği).
 2. P'nin sağ alt ağacının sağına eklendiğinde (RR Dengesizliği)
 - **İç Durum** (2 kez döndürme işlemi gerektiren) :
 3. P'nin sol alt ağacının sağına eklendiğinde (RL Dengesizliği)
 4. P'nin sağ alt ağacının soluna eklendiğinde (LR Dengesizliği)

LL Dengesizliği & Düzeltme



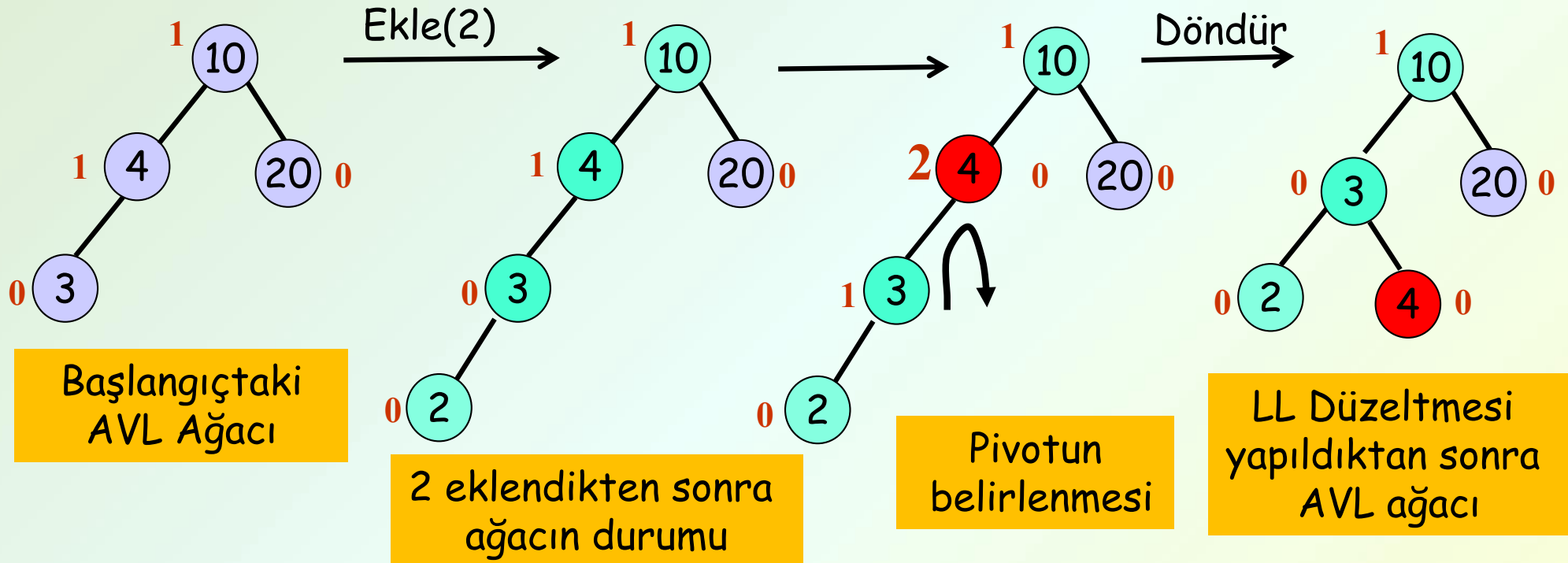
Ekleme işleminden sonra ağaç



LL Düzeltmesinden sonra ağaç

- **LL Dengesizliği:** P'nin sol alt ağacının soluna eklendiğimizde (A alt ağacına)
 - P'nin bfsi 2
 - L'nin bfsi 0 veya 1
- **Düzeltme:** P etrafında sağa doğru dönderme.

LL Dengesizliği Düzeltme Örneği (1)

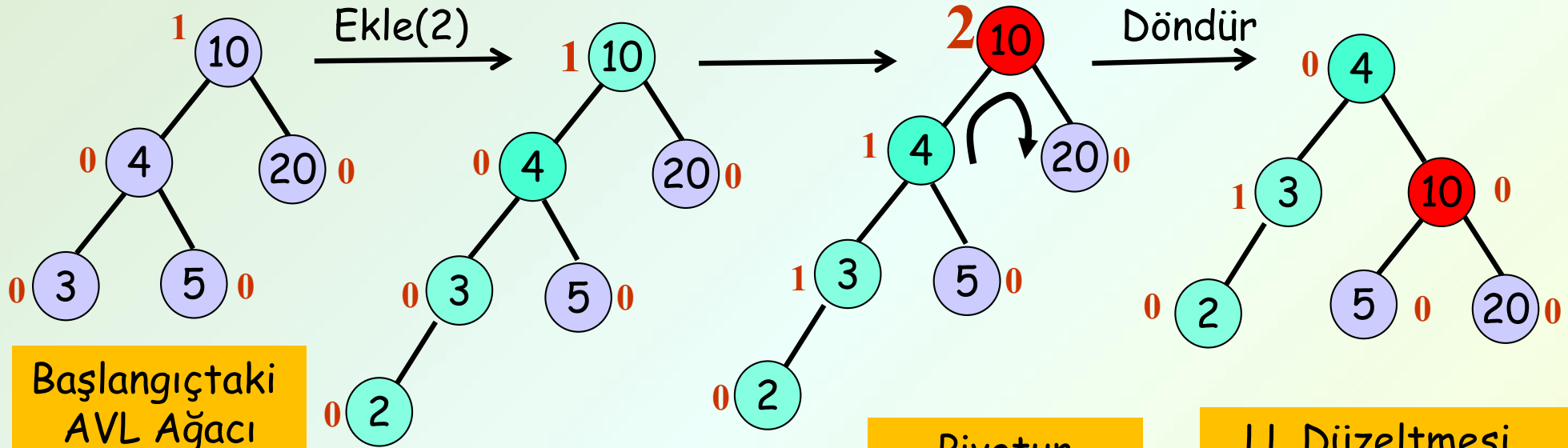


Balans faktörü
düzelterek köke
doğru ilerle

Dengesizliğin türünün
belirlenmesi

- LL Dengesizliği:
 - P(4)'ün bfsi 2
 - L(3)'ün bfsi 0 veya 1

LL Dengesizliği Düzeltme Örneği (2)



Başlangıçtaki
AVL Ağacı

2 eklendikten sonra
ağacın durumu

Balans faktörü
düzelterek köke
doğru ilerle

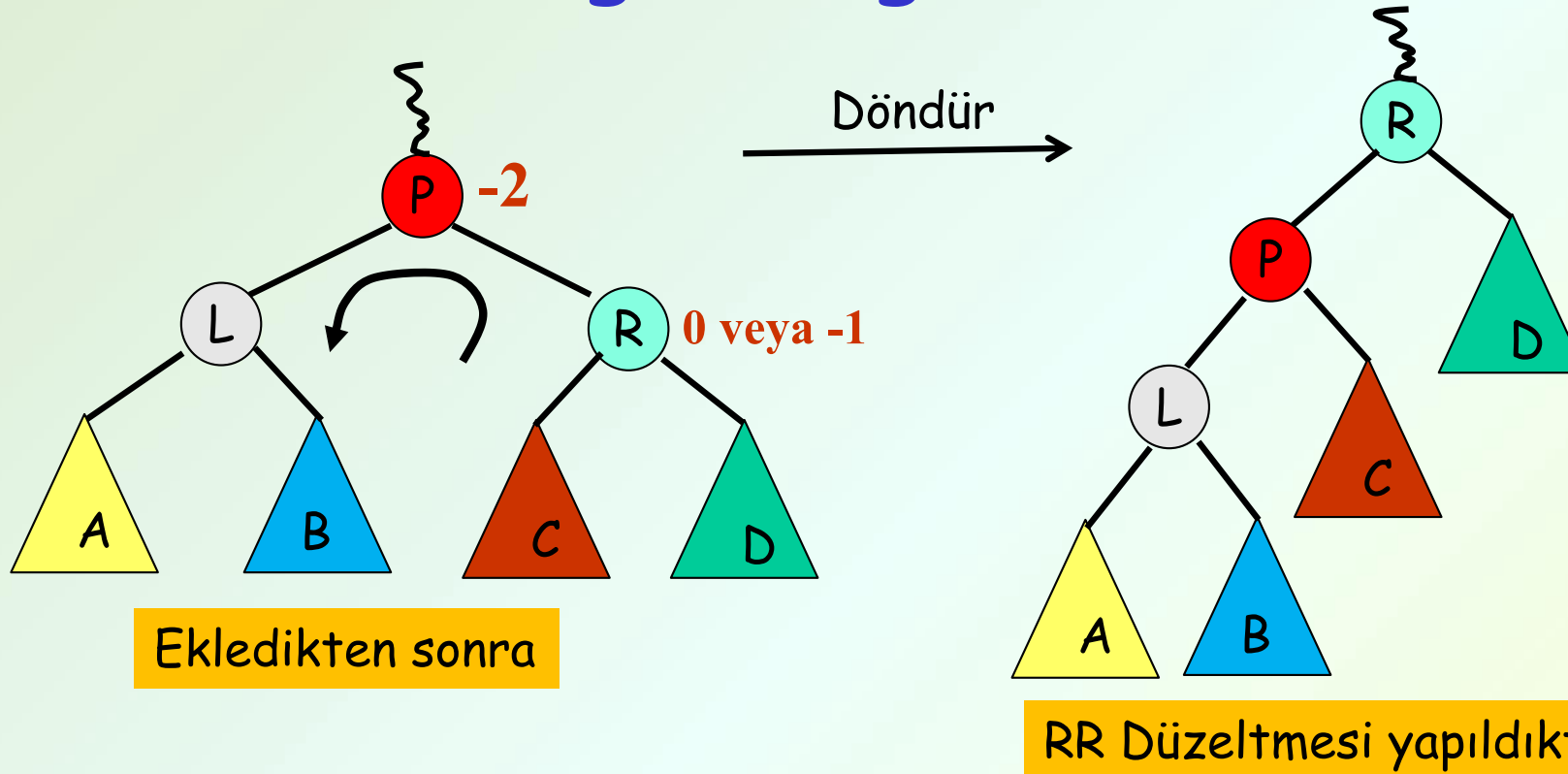
Pivotun
belirlenmesi

Dengesizliğin
türünün
belirlenmesi

LL Düzeltmesi
yapıldıktan sonra
AVL ağacı

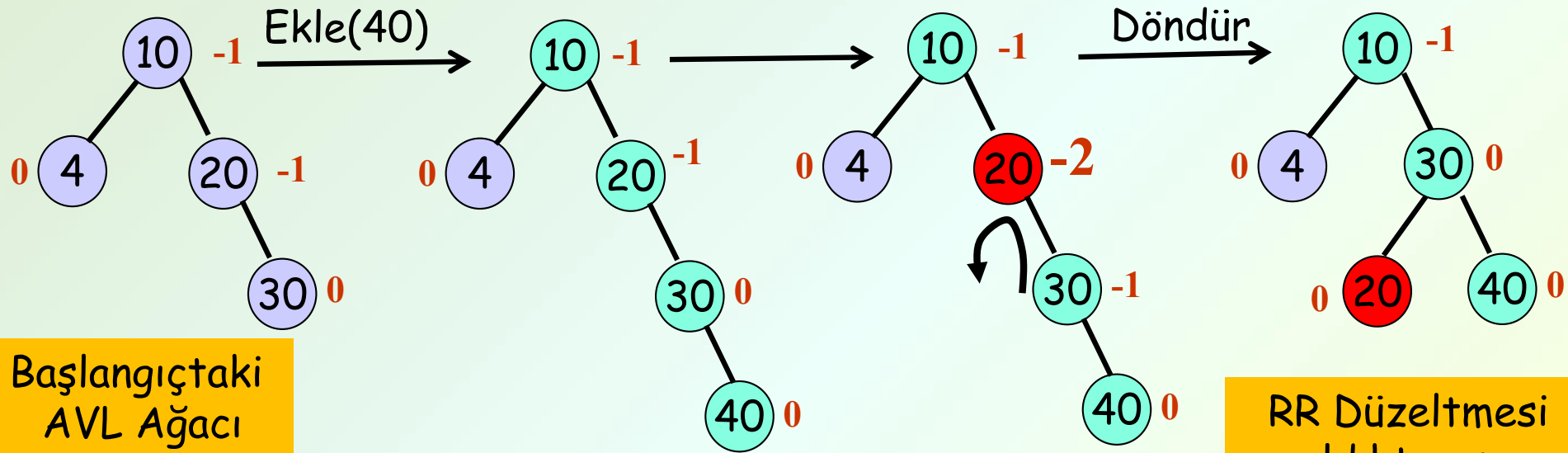
- LL Dengesizliği:
 - P(4)'ün bfsi 2
 - L(3)'ün bfsi 0 veya 1

RR Dengesizliği & Düzeltme



- **RR Dengesizliği:** P'nin sağ alt ağacının sağına eklendiğinde (D alt ağacına eklendiğinde)
 - $P \rightarrow bf = -2$
 - $R \rightarrow bf = 0 \text{ veya } -1$
- **Düzeltme:** P etrafında sola doğru dönderme

RR Dengesizliği Düzeltme Örneği (1)



Başlangıçtaki
AVL Ağacı

40 eklendikten sonra
ağacın durumu

Pivotun
belirlenmesi

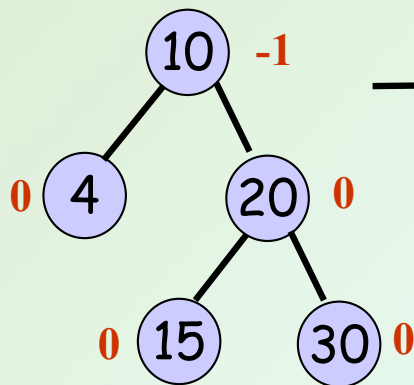
RR Düzeltmesi
yapıldıktan sonra
AVL ağacı

Balans faktörü
düzelterek köke
doğru ilerle

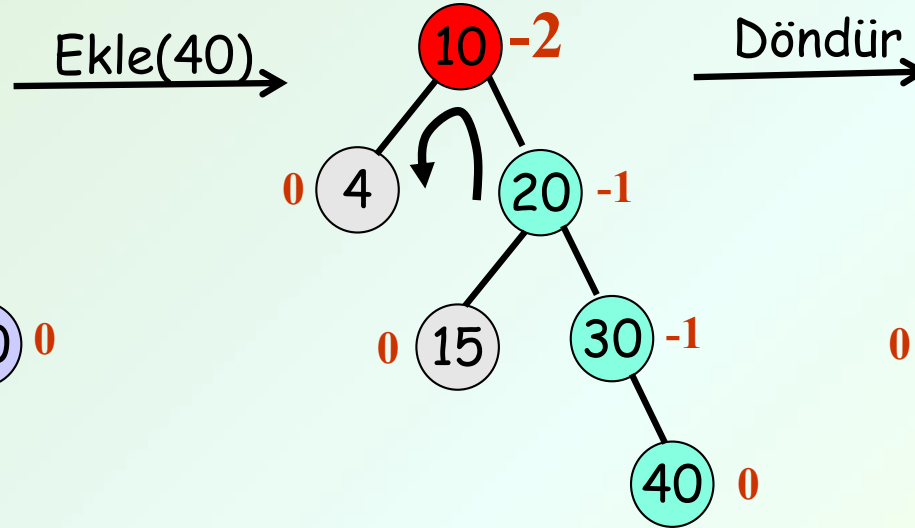
Dengesizliğin türünün
belirlenmesi

- **RR Dengesizliği:**
 - $P(20) \rightarrow bf = -2$
 - $R(30) \rightarrow bf = 0$ veya -1

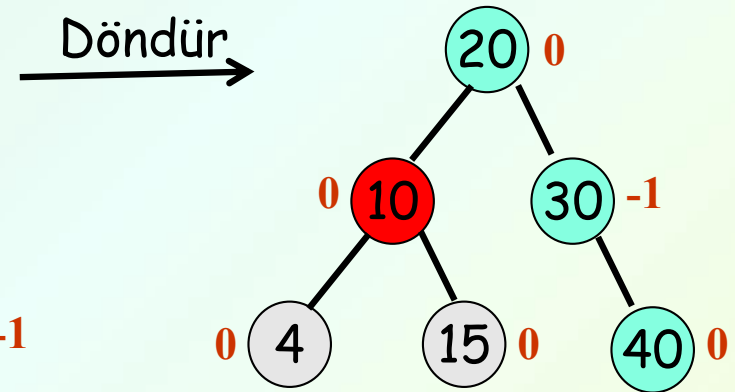
RR Dengesizliği Düzeltme Örneği (2)



Başlangıçtaki
AVL Ağacı



40 eklendikten sonra
ağacın durumu



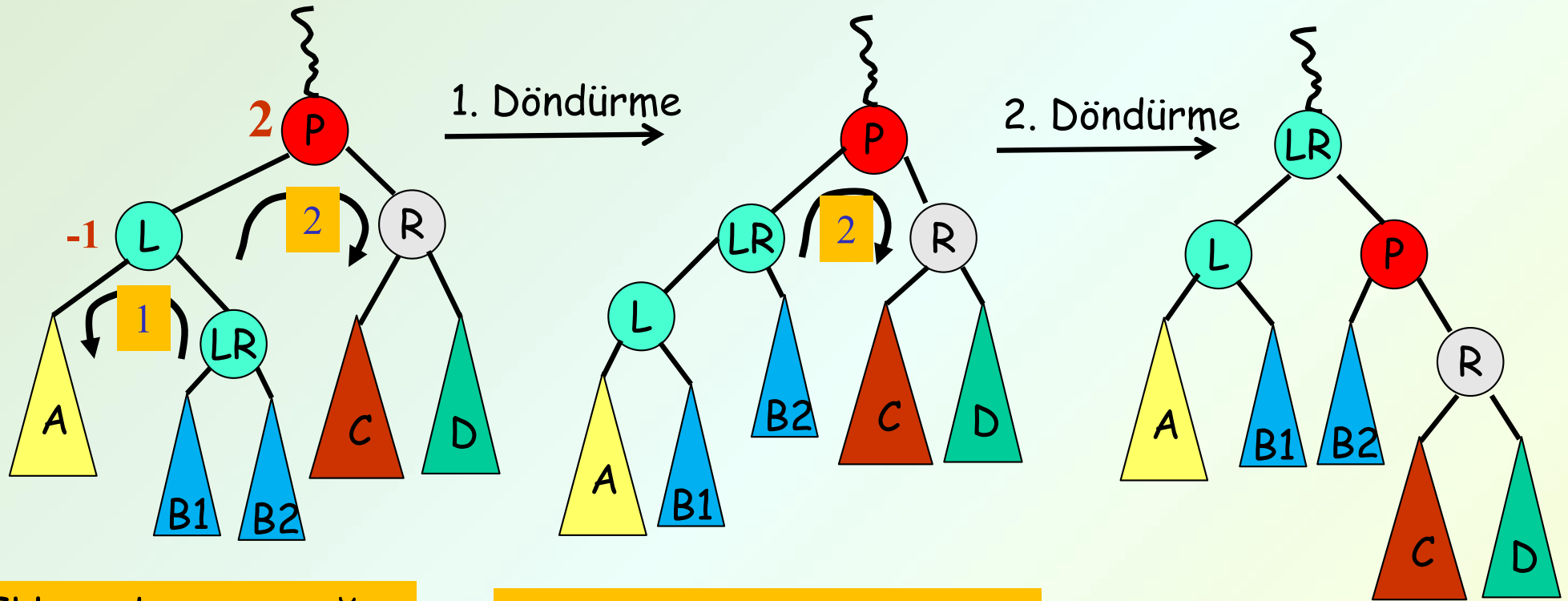
RR Düzeltmesi
yapıldıktan sonra
AVL ağacı

Balans faktörü
düzelterek köke doğru
ilerle ve 10'u pivot
olarak belirle

Dengesizliğin türünün
belirlenmesi

- **RR Dengesizliği:**
 - $P(10) \rightarrow bf = -2$
 - $R(20) \rightarrow bf = 0$ veya -1

LR Dengesizliği & Düzeltme



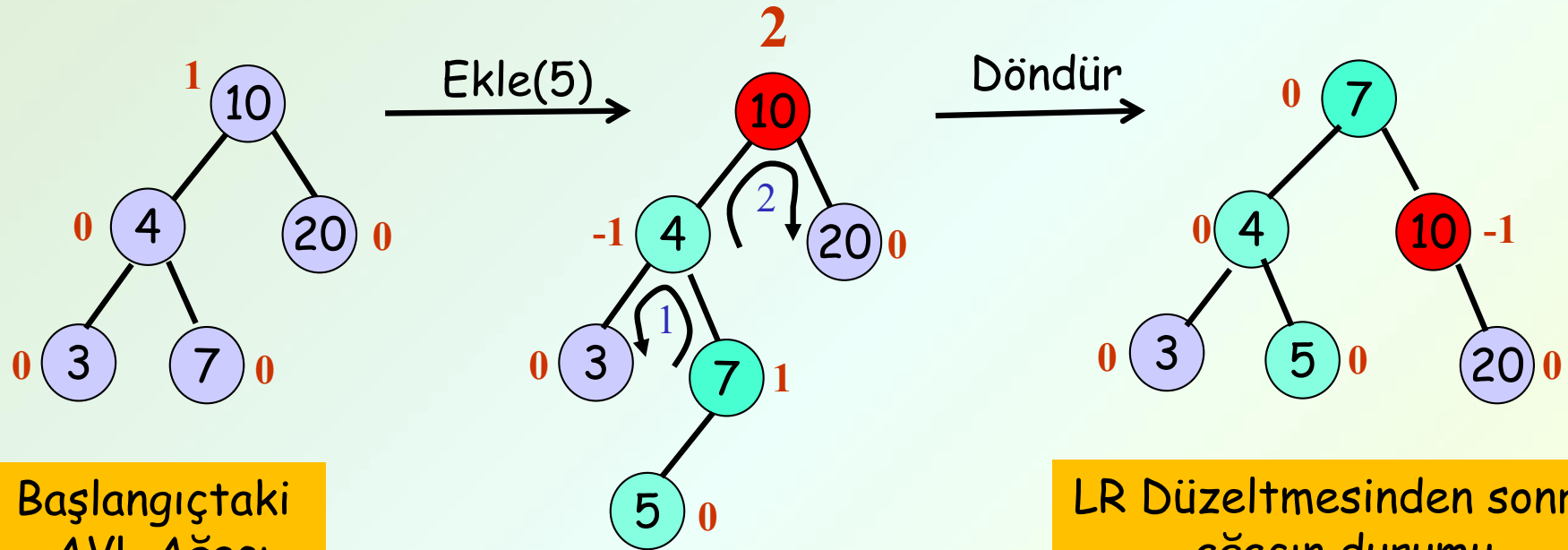
Eklemeden sonra ağaç

1. Döndürmeden sonra ağaç

LR Düzeltmesinden sonra

- **LR Dengesizliği:** P'nin sol alt ağacının sağına eklendiğinde (**LR ağacına**)
 - $P \rightarrow bf = 2$
 - $L \rightarrow bf = -1$
- **Düzeltme:** L & P etrafında 2 kez döndürme

LR Dengesizliği Düzeltme Örneği



Başlangıçtaki
AVL Ağacı

LR Düzeltmesinden sonra
ağacın durumu

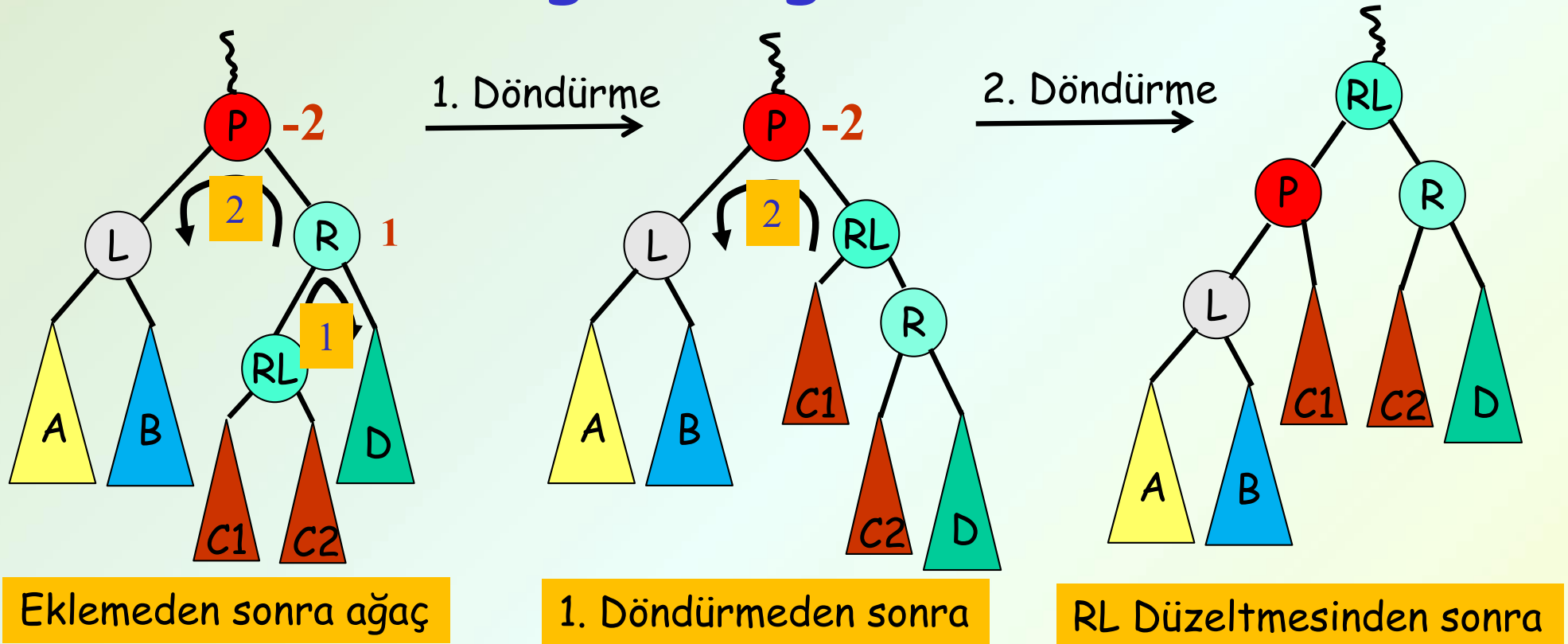
2 eklendikten sonra
ağacın durumu

Dengesizliğin türünün
belirlenmesi

Balans faktörü
düzelterek köke doğru
ilerle ve 10'u pivot
olarak belirle

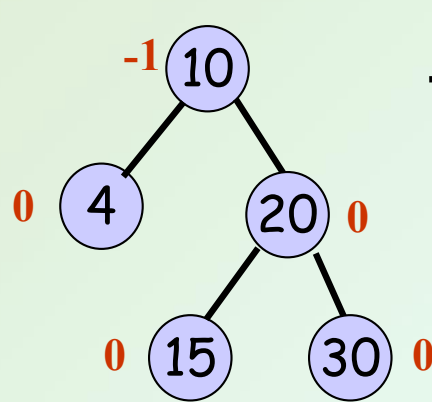
- **LR Dengesizliği:**
 - $P(10) \rightarrow bf = 2$
 - $L(4) \rightarrow bf = -1$

RL Dengesizliği & Düzeltme



- **RL Dengesizliği:** P'nin sağ alt ağacının soluna eklendiğinde (RL alt ağacına)
 - $P \rightarrow bf = -2$
 - $R \rightarrow bf = 1$
- **Düzeltme:** R & P etrafında 2 kez döndürme.

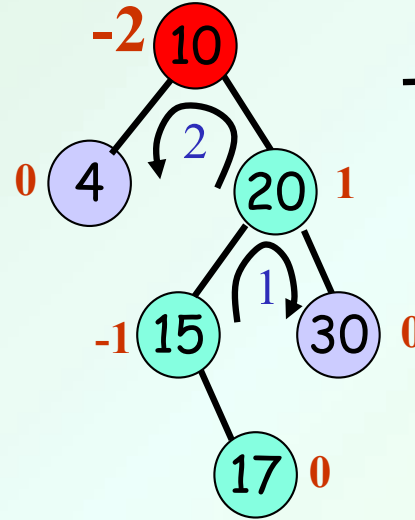
RL Dengesizliği Düzeltme Örneği



Başlangıçtaki
AVL Ağacı

Balans faktörü
düzelterek köke
doğru ilerle ve 10'u
pivot olarak belirle

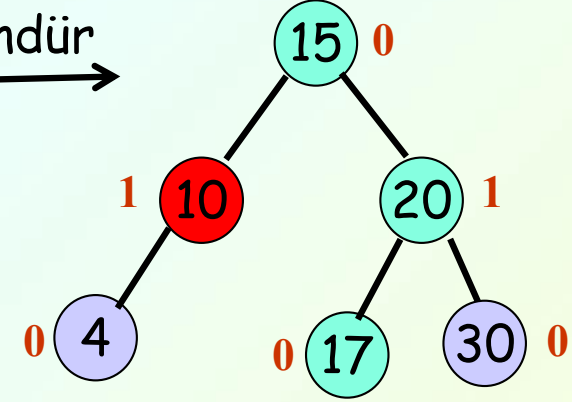
Ekle(17)



17 eklendikten sonra
ağacın durumu

Dengesizliğin türünün
belirlenmesi

Döndür



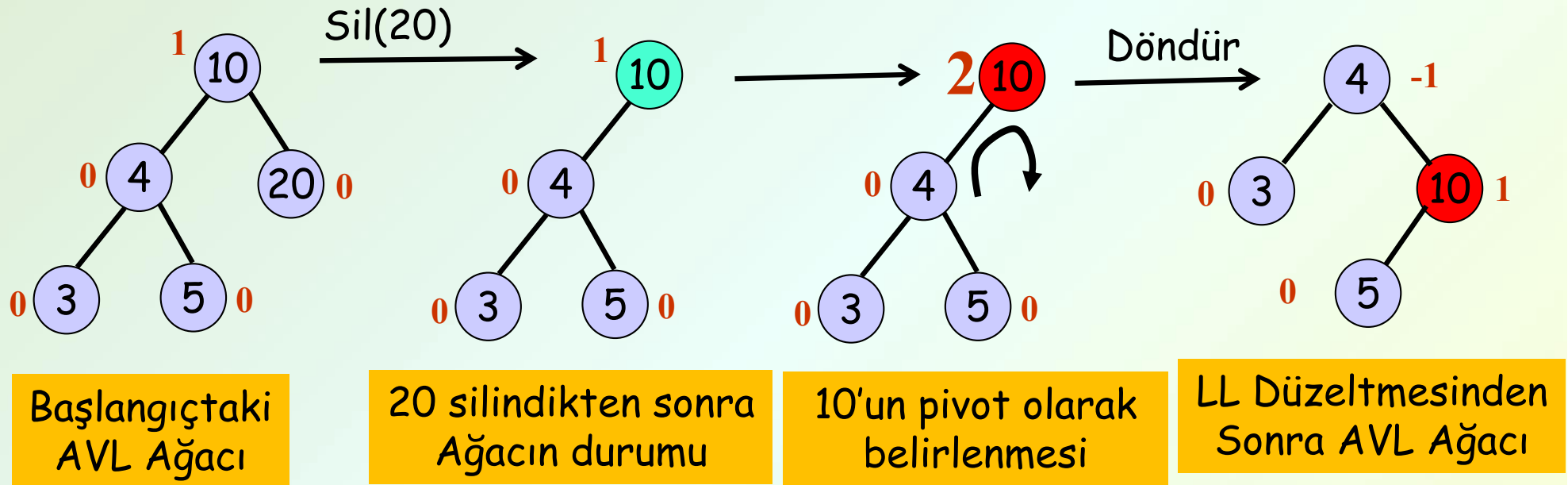
RL Düzeltmesi
yapıldıktan sonra
AVL ağacı

- **RL Dengesizliği:**
 - $P(10) \rightarrow bf = -2$
 - $R(20) \rightarrow bf = 1$

Silme

- Silme işlemi ekleme işlemi ile benzerlik gösterir.
- Öncelikle ikili arama ağacındaki gibi normal silme işlemi yapılır.
- Düğüm silindikten sonra, ağaçta köke doğru ilerleyin ve balans faktörünü güncelleyin.
 - Eğer dengesiz bir düğüm tespit edilirse, uygun döndürme işlemlerini yapın.
 - Bu işlem sırasında birden fazla döndürme işlemi yapılabilir.

Silme Örneği (1)

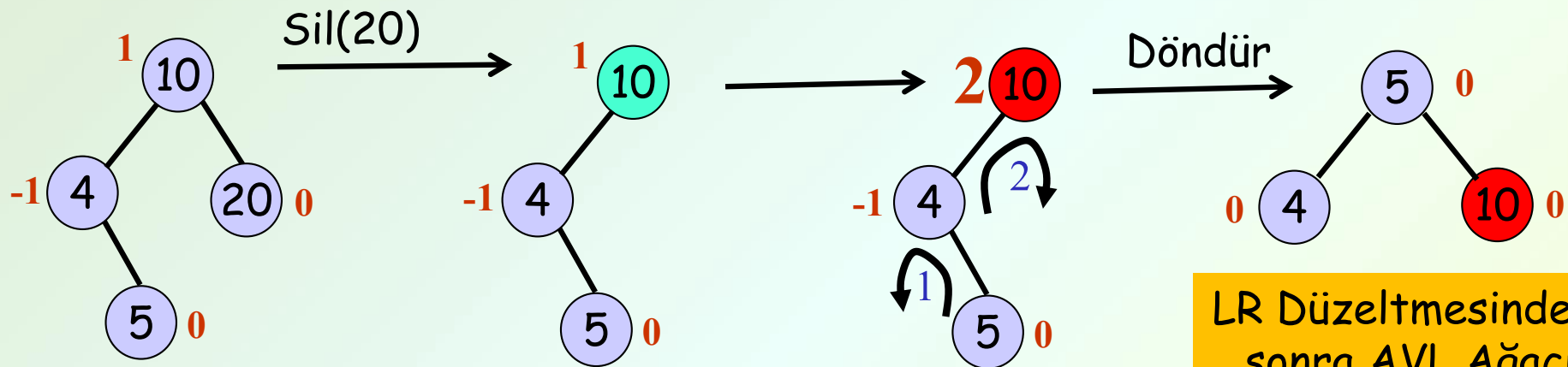


Balans faktörü düzelterek köke doğru ilerle

Dengesizliğin türünün belirlenmesi

- LL Dengesizliği:
 - P(10)'ün bfsi 2
 - L(4)'ün bfsi 0 veya 1

Silme Örneği (2)



Başlangıçtaki
AVL Ağacı

20 silindikten sonra
Ağacın durumu

10'un pivot olarak
belirlenmesi

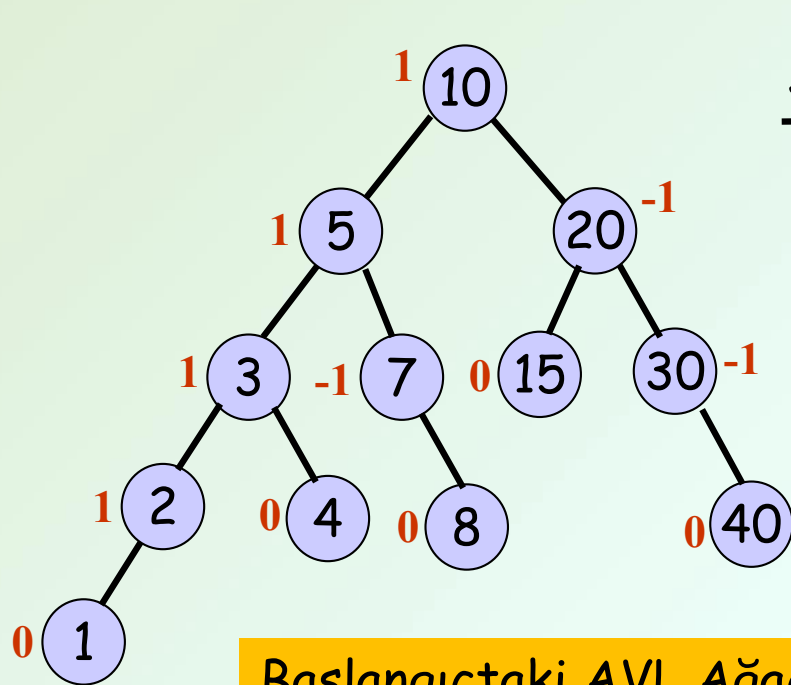
LR Düzeltmesinden
sonra AVL Ağacı

Balans faktörünü
düzelterek köke
doğru ilerle

Dengesizliğin
türünün belirlenmesi

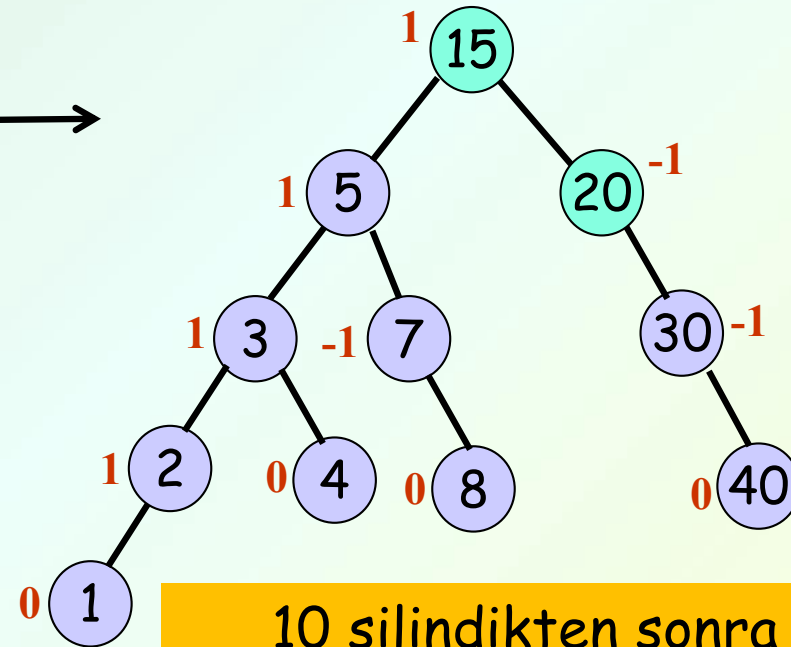
- LR Dengesizliği:
 - $P(10) \rightarrow bf = 2$
 - $L(4) \rightarrow bf = -1$

Silme Örneği (3)



Başlangıçtaki AVL Ağacı

Sil(10) →

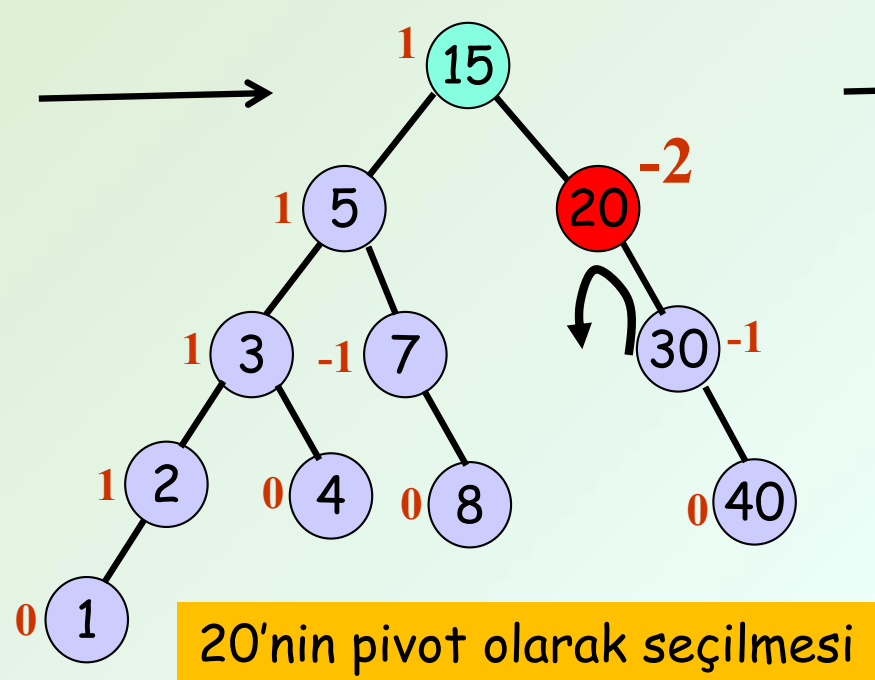


10 silindikten sonra

10 ve 15(sağ alt ağaçtaki en küçük eleman) yer değiştirdi ve 15 silindi.

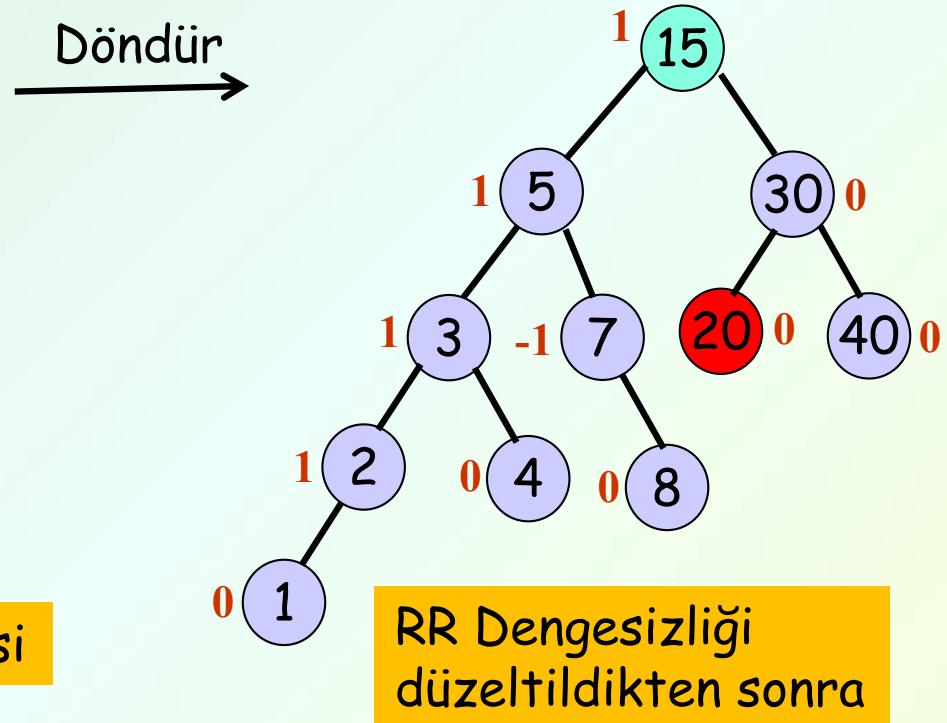
Balans faktörü düzelterek köke doğru ilerle

Silme örneği (3) - devam



Dengesizliğin türünü belirle

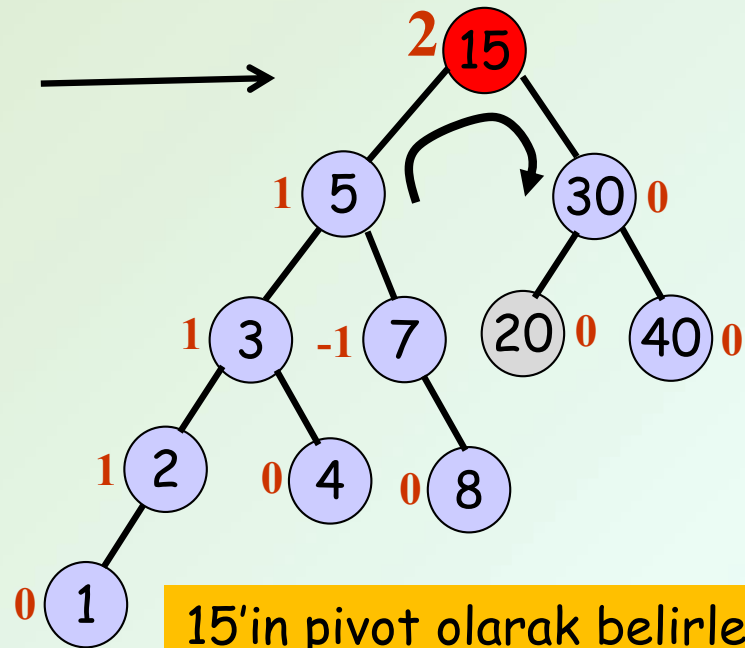
- **RR Dengesizliği:**
 - $P(20) \rightarrow bf = -2$
 - $R(30) \rightarrow bf = 0$ veya -1



Yukardaki AVL ağacı mıdır?

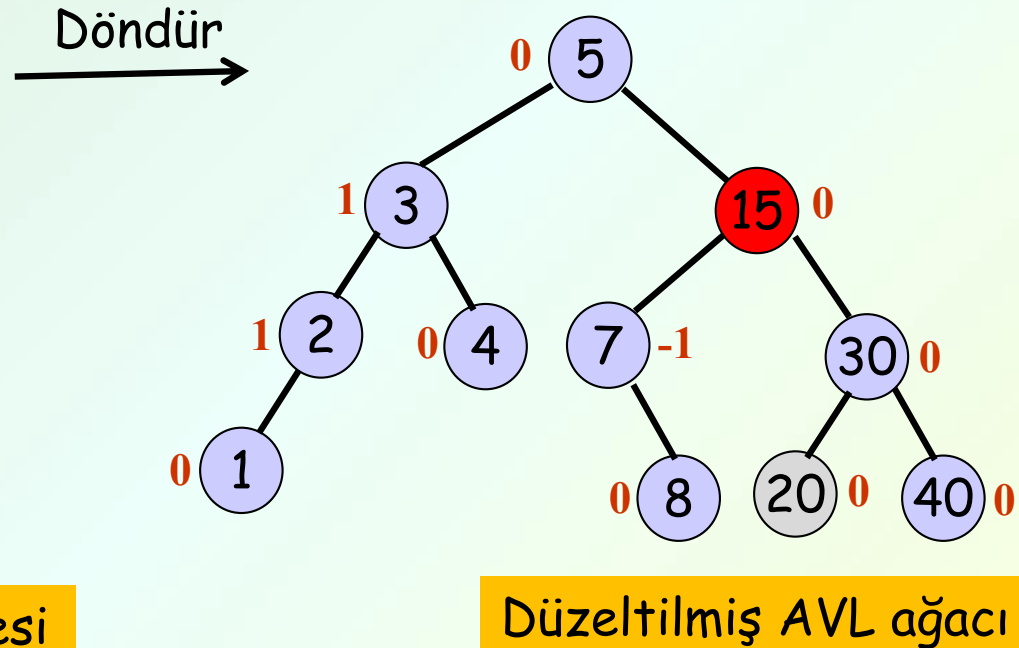
Balans faktörü düzelterek köke doğru ilerle.

Silme Örneği (3) - devam



Dengesizliğin türünü belirle

- LL Dengesizliği:
 - $P(15) \rightarrow bf = 2$
 - $L(5) \rightarrow bf = 0$ veya 1



Arama (Bul)

- AVL ağacı bir tür İkili Arama Ağacı (BST) olduğundan arama algoritması BST ile aynıdır.
- $O(\log N)$ de çalışması garantidir.

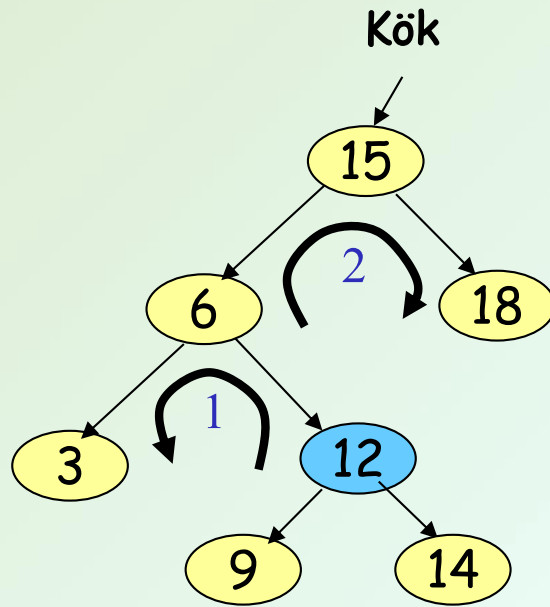
AVL Ağaçları - Özet

- AVL Ağaçlarının Avantajları
 1. AVL ağacı devamlı dengeli olduğu için Arama/Ekleme/Silme işlemi $O(\log N)$ yapılır.
 2. Düzeltme işlemi algoritmaların karmaşıklığını etkilemez.
- AVL Ağaçlarının Dezavantajları:
 1. Balans faktörü için ekstra yer gereklidir.
 2. Arı dengeli ağaçlarda AVL ağaçları kadar iyi performans verir ayrıca bu tür algoritmalarda balans faktör kullanılmaz
 - Splay ağaçları

Splay Ağaçları

- Splay ağaçları ikili arama ağaç yapılarından birisidir.
 - Her zaman dengeli değildir.
 - Arama ve ekleme işlemlerinde dengelemeye çalışılır böylece gelecek operasyonlar daha hızlı çalışır.
- Sezgiseldir.
 - Eğer X bir kez erişildi ise, bir kez daha erişilecektir.
 - X erişildikten sonra "splaying" operasyonu yapılır.
 - X köke taşınır.

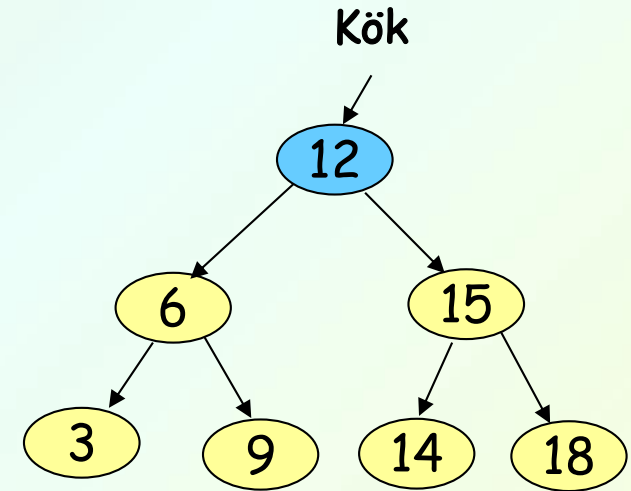
Örnek



Başlangıçtaki ağaç

Bul(12) den sonra

Splay ana düşünce:
Döndürme kullanarak
12'yi köke getir.

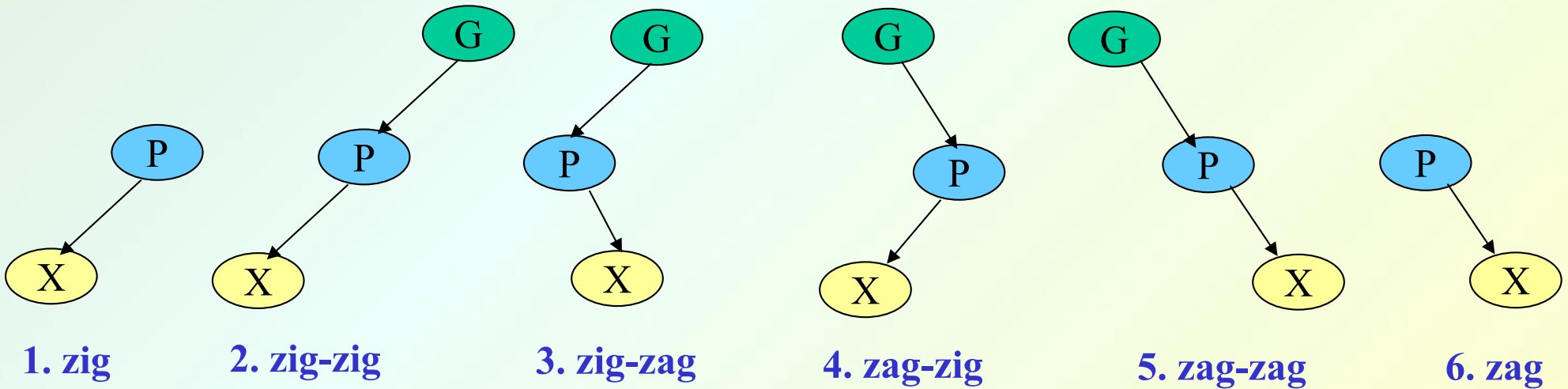


12 arandıktan sonra

- Burada ağacın dengeli olmasının yanı sıra 12 ye $O(1)$ zamanında erişilebiliyor.
- **Aktif (son erişilenler)** düğümler köke doğru taşınırken aktif olmayan düğümler kökten uzaklaştırılır.

Splay Ağacı Terminolojisi

- X kök olmayan bir düğüm olsun. (ailesi olan)
- P düğümü X'in aile düğümü olsun.
- G düğümü X'in ata düğümü olsun.
- **G ve X** arasındaki yol düşünüldüğünde:
 - Her sola gitme işlemine "**zig**" işlemi denilmektedir.
 - Her sağa gitme işlemine "**zag**" işlemi denilmektedir.
- Toplam 6 farklı durum oluşabilir:

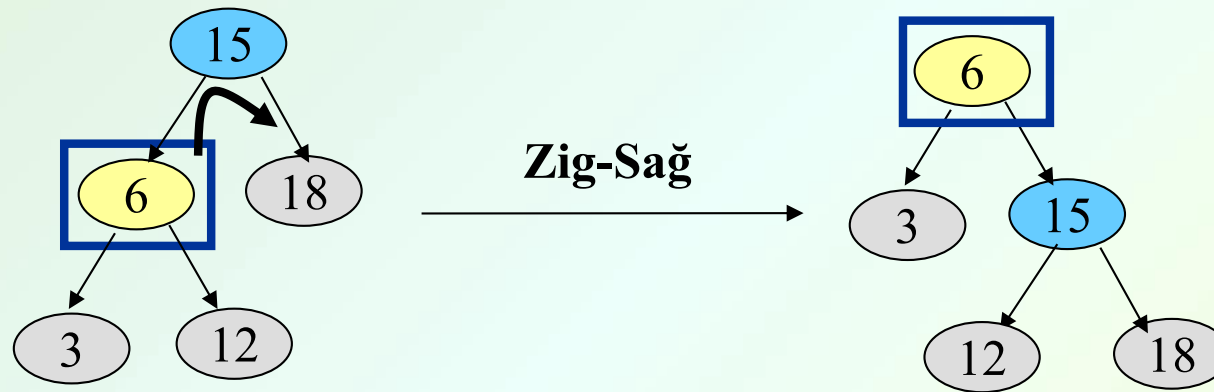


Splay Ağaç İşlemleri

- X'e erişildiğinde 6 tane rotasyon işleminden birisi uygulanır:
 - Tek Dönme (X P'ye sahip ama G'ye sahip değil)
 - zig, zag
 - Çift Dönme (X hem P hemde G'ye sahip)
 - zig-zig, zig-zag
 - zag-zig, zag-zag

Splay Ağaçları: Zig İşlemi

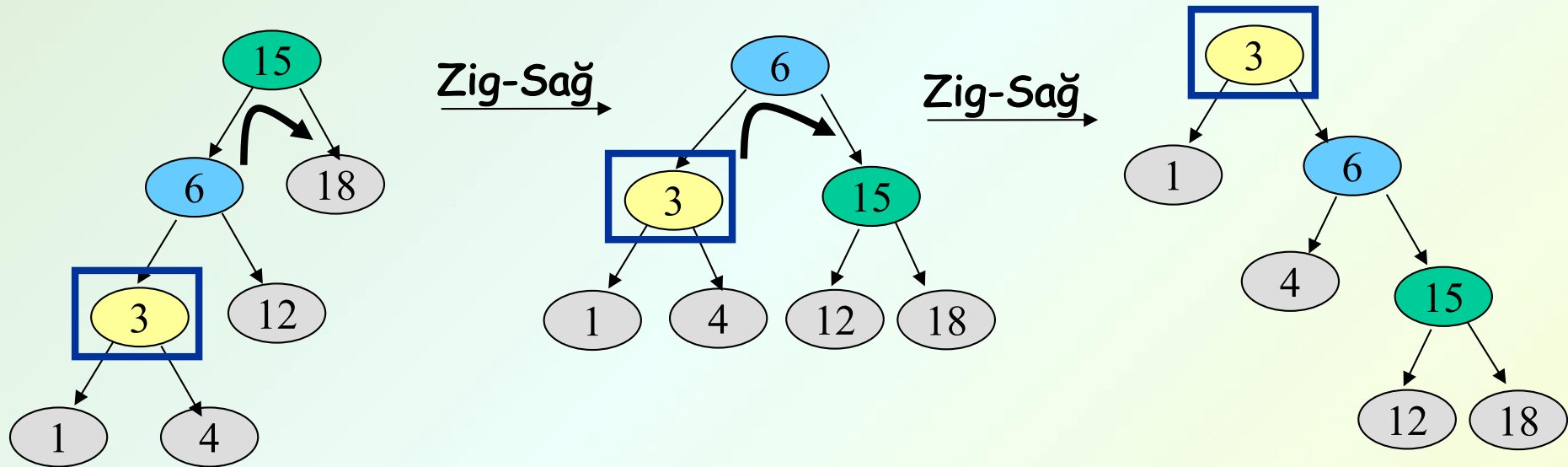
- “Zig” işlemi AVL ağacındaki gibi tek döndürme işlemidir.
- Örneğin erişilen elemanın 6 olduğu düşünülürse.



- “Zig-Sağ” işlemi 6’yı köke taşır.
- Bir sonraki işlemde 6’ya $O(1)$ de erişilebilir.
- AVL ağacındaki sağ dönme işlemi ile benzerdir.

Splay Ağaçları: Zig-Zig İşlemi

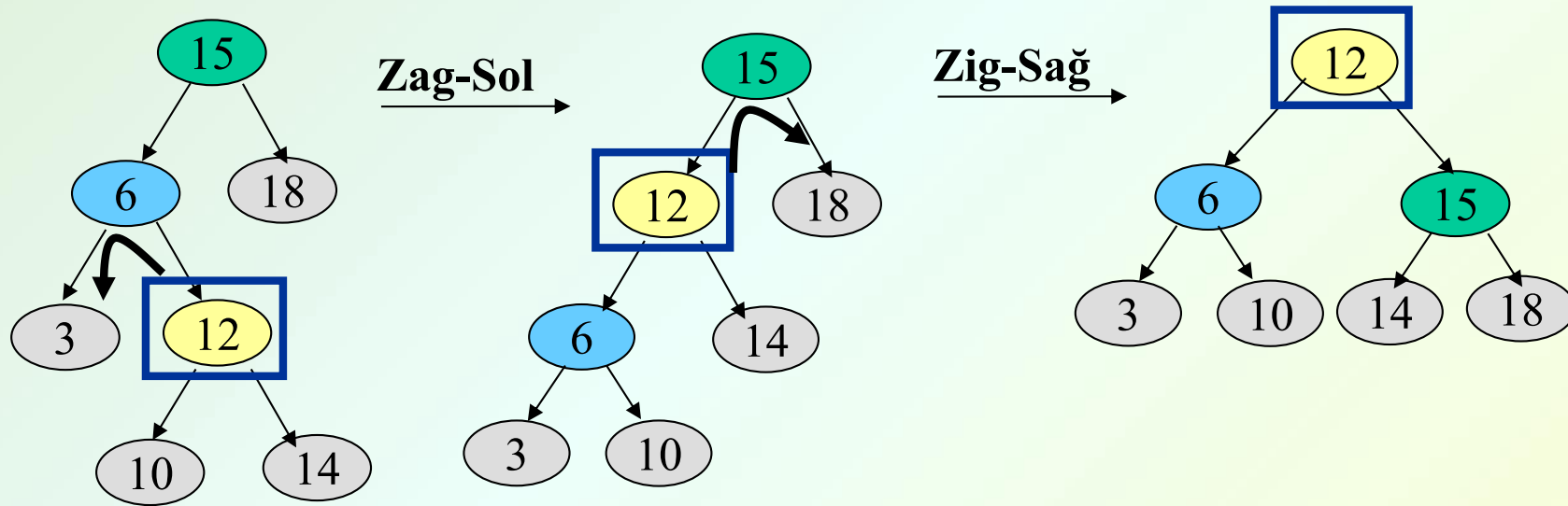
- “Zig-Zig” işlemi aynı türde 2 tane dönme işlemi içerir.
- Örneğin erişilen elemanın 3 olduğu düşünülürse.



- “zig-zig” işlemi ile birlikte 3 köke taşınmış oldu.
- Not: Aile - ata arasındaki döndürme önce yapılıyor.

Splay Ağaçları: Zig-Zag İşlemi

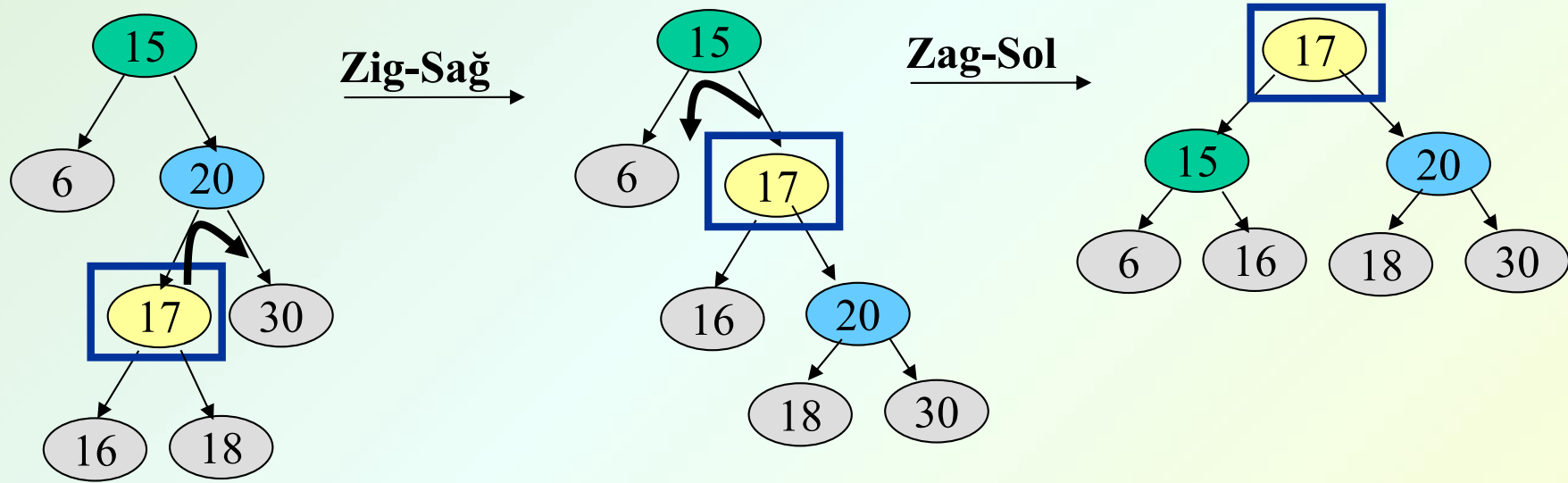
- “Zig-Zag” işlemi farklı türde 2 tane dönme işlemi içerir.
- Örneğin erişilen elemanın 12 olduğu düşünülürse.



- “zig-zag” işlemi ile birlikte 12 köke taşınmış oldu.
- AVL ağacındaki **LR dengesizliğini** düzeltmek için kullanılan işlemler ile aynıdır.(önce sol dönme, daha sonra sağ dönme)

Splay Ağaçları: Zag-Zig İşlemi

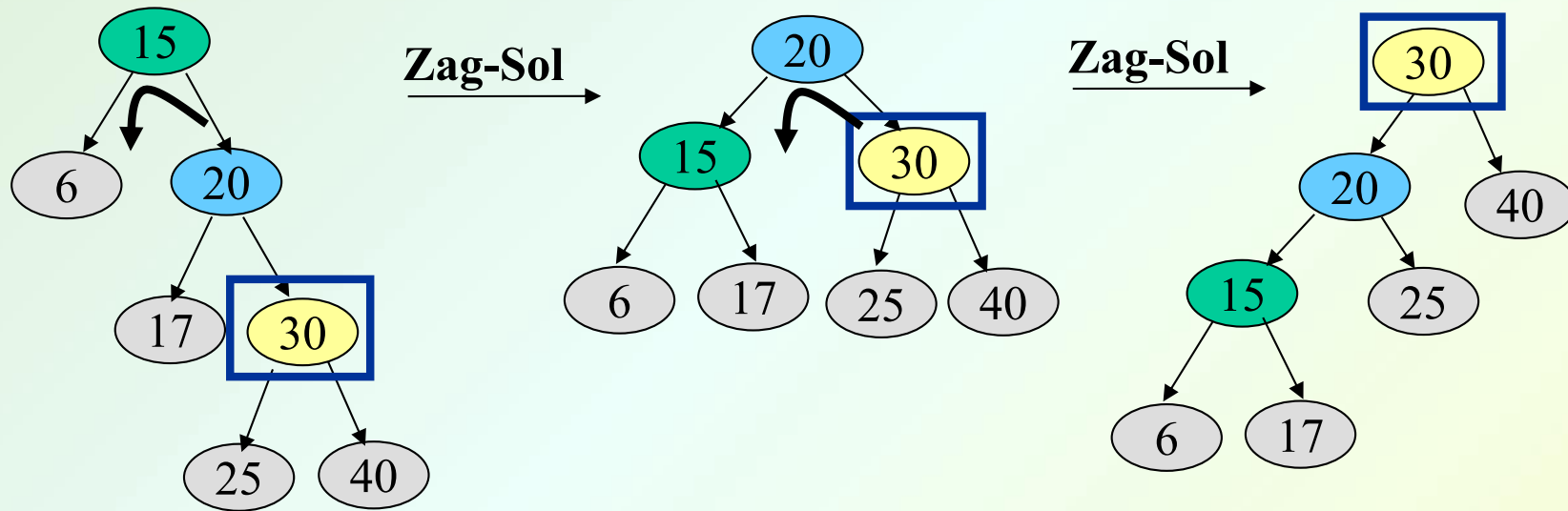
- “Zag-Zig” işlemi farklı türde 2 tane dönme işlemi içerir.
- Örneğin erişilen elemanın 17 olduğu düşünülürse.



- “zag-zig” işlemi ile birlikte 17 köke taşınmış oldu.
- AVL ağacındaki **RL dengesizliğini** düzeltmek için kullanılan işlemler ile aynıdır.(önce sağ dönme, daha sonra sol dönme)

Splay Ağaçları: Zag-Zag İşlemi

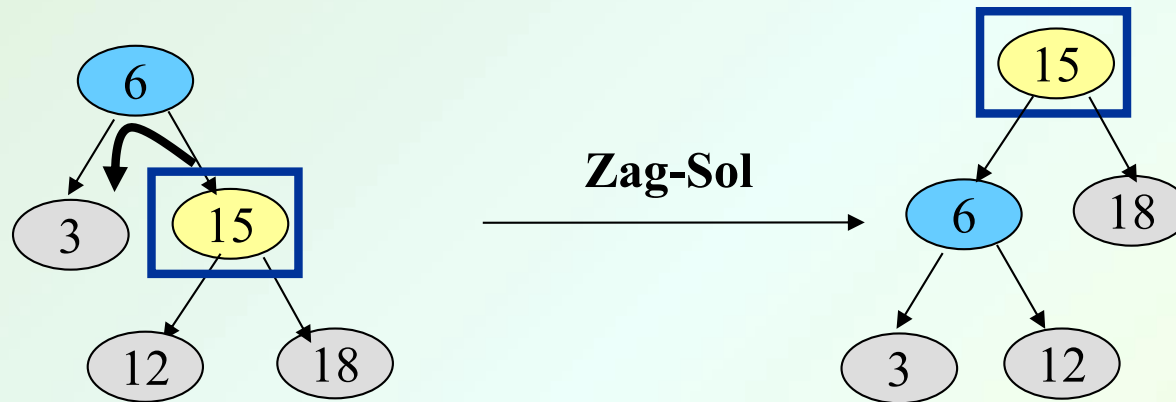
- “Zag-Zag” işlemi aynı türde 2 tane dönme işlemi içerir.
- Örneğin erişilen elemanın 30 olduğu düşünülürse.



- “zag-zag” işlemi ile birlikte 30 köke taşınmış oldu.
- Not: Aile - ata arasındaki döndürme önce yapılıyor.

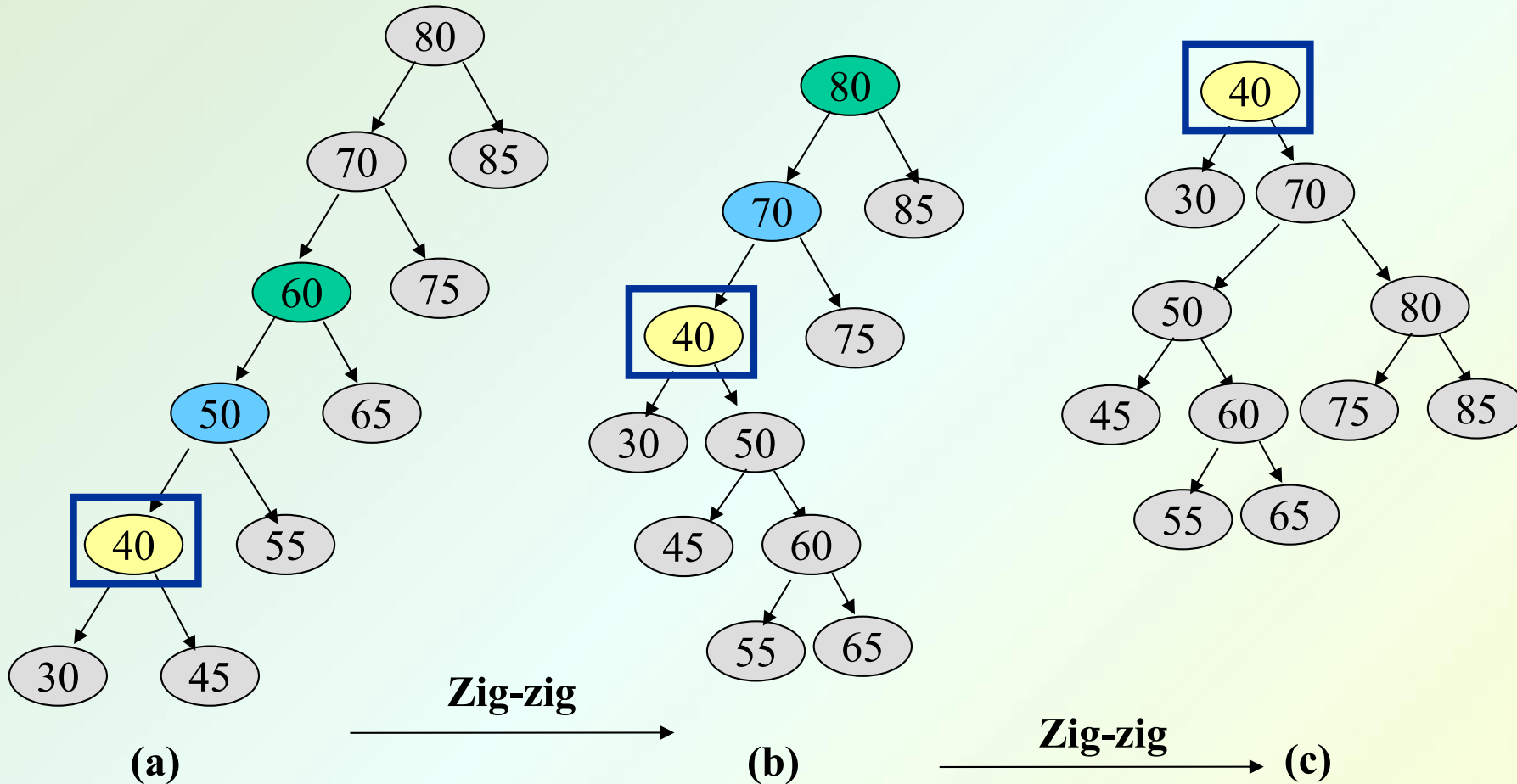
Splay Ağaçları: Zag İşlemi

- “Zag” işlemi AVL ağacındaki gibi tek döndürme işlemidir.
- Örneğin erişilen elemanın 15 olduğu düşünülürse.

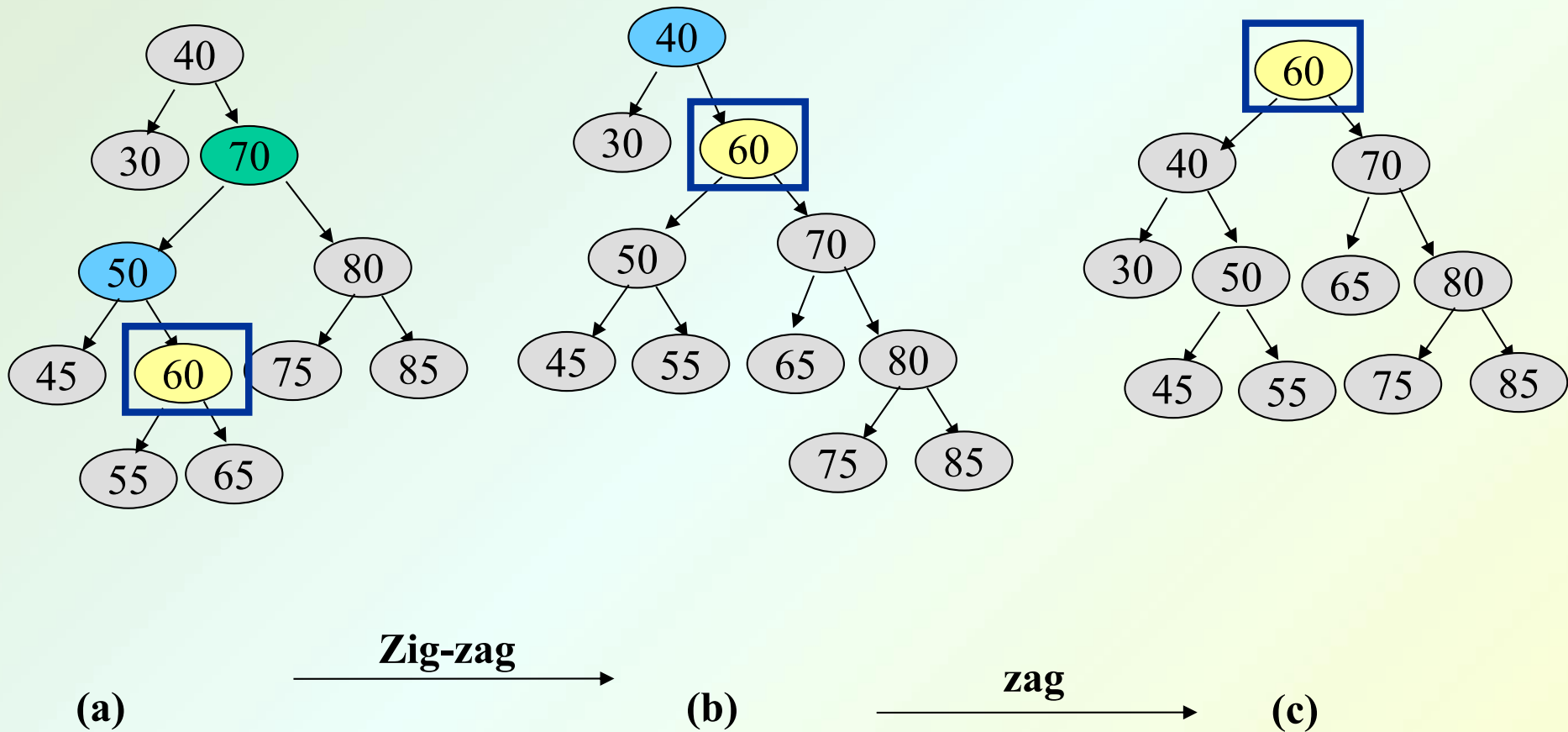


- “Zag-sol işlemi 15’i köke taşır.
- Bir sonraki işlemde 15’e $O(1)$ de erişilebilir.
- AVL ağacındaki sol dönme işlemi ile benzerdir.

Splay Ağacı Örnek: 40'a Erişildiğinde



Splay Ağacı Örnek: 60'a Erişildiğinde



Diğer işlemler Sırasında Splaying

- Splaying sadece Arama işleminden sonra değil Ekle/Sil gibi değer işlemlerden sonra da uygulanabilir.
- **Ekle X**: X yaprak düğüme eklendikten sonra (BST işlemi) X'i köke taşı.
- **Sil X**: X'i ara ve köke taşı. Kökte bulunan X'i sil ve sol alt ağaçtaki en büyük elemanı köke taşı.
- **Bul X**: Eğer X bulunamazsa aramanın sonlandığı yaprak düğümü köke taşı.

Splay Ağaçları - Özet

- Splaying işlemi ile ağaç genel olarak dengede kalıyor.
- **Analiz Sonucu:** N boyutlu bir Splay ağacı üzerinde M tane işlemin çalışma süresi $O(M \log N)$ dir. Dolayısıyla tek bir işlem için çalışma zamanı $O(\log N)$ dir.
- Erişilecek elemanların derinliği çok büyük olsa bile, arama işlemlerinin süresi bir süre sonra kısalmaktadır. Çünkü her bir arama işlemi ağacın dengelenmesini sağlıyor.

Örnek - 1

- 1, 2, ..., 6 elemanları sırası ile boş bir Splay ağacına ekleniyor.
- Oluşan ağacı çizerek gösteriniz.
- "6" ya erişim nasıl olur.
- "1" e erişim sağlandıktan sonra ağacın yapısını çizerek gösteriniz.

Örnek - 2

- 1, 2, ..., 6 elemanları sırası ile boş bir AVL ağacına ekleniyor.
- Her bir eleman eklendiğinde oluşan ağaçları çizerek gösteriniz.
- "6" ya erişim nasıl olur.